

HOMEWORK 4
ME 326: Collaborative Robotics
Stanford University
Winter 2026

Due: 11:59 pm, March 1, 2026

Worth: 35pts

Turn in to Canvas:

1. PDF of Jupyter notebook
2. Completed Jupyter notebook file

1 Problem: LSTM Sequence Prediction [10pts]

Note: *Figures in this problem are referenced/imported from this toward data science article: [Illustrated Guide to LSTM's and GRU's: A step by step explanation](#) (and is encouraged for further reading), also checkout this [video](#) for a video summary of how LSTM's work.*

Sequential Data In some problems, it is beneficial to recognize a pattern that emerges from a sequence of data. If there are underlying principles driving behavior, they may be best interpreted by observing the system over time then using those observations to extrapolate to expected future sequence states.

This leads to the study of **Recurrent Neural Network**(RNN) which are designed specifically to work with sequential data (could be time dependent or time independent). The key components of the general architecture of a RNN is the input vector x (which was observed or known) being fed into the RNN, and the RNN maintains *hidden* states h which serve as "memory" for the network as it processes new input and/or predicts future outputs. A key challenge in such a network is the question: "*what information is most important to remember?*"

Long-Short Term Memory (LSTM) networks (a type of RNN) are one answer to this question, with the ability to *forget*, *input new information into memory*, provide both a *new output (hidden state)* available to external query as well as *maintain an updated internal state (cell state)* for future memory iterations.

While it will not be discussed further in this homework, the student should be aware of another option - [Attention Networks](#) which observe a sequence and determine what is the most important information to *pay attention to* in order to make some prediction or estimation.

For the LSTM, the structure is shown here and consists of a forget gate, cell state, input gate and output gate. For completeness, a Gated Recurrent Unit (GRU) is shown (very similar to LSTM with a few less 'control' handles - it is the *newer* method often used due to its slight simplicity yet equivalent capability of LSTM).

A basic RNN cell takes in a hidden state and input state, passes it through a Tanh function and outputs a new hidden state (where the Tanh activation serves to map the values between -1 and 1).

For the LSTM shown in Figure 1, the key attribute of its architecture is its various gates. The *cell state* is designed to carry relevant information throughout iterations of the sequence serving as a 'transport highway' of information. As the cell state progresses, it must be informed by new information and focus on what is important through the use of *gates*. The *gates* leverage sigmoid activation functions which compress continuous input information between 0 and 1.

The different gates are as follows:

Forget Gate The *forget gate* decides which information should be 'thrown away' or kept and this is done by passing added previous h and current x information through a sigmoid function where values closer to 0 implies to *forget* and values closer to 1 indicate to *keep*.

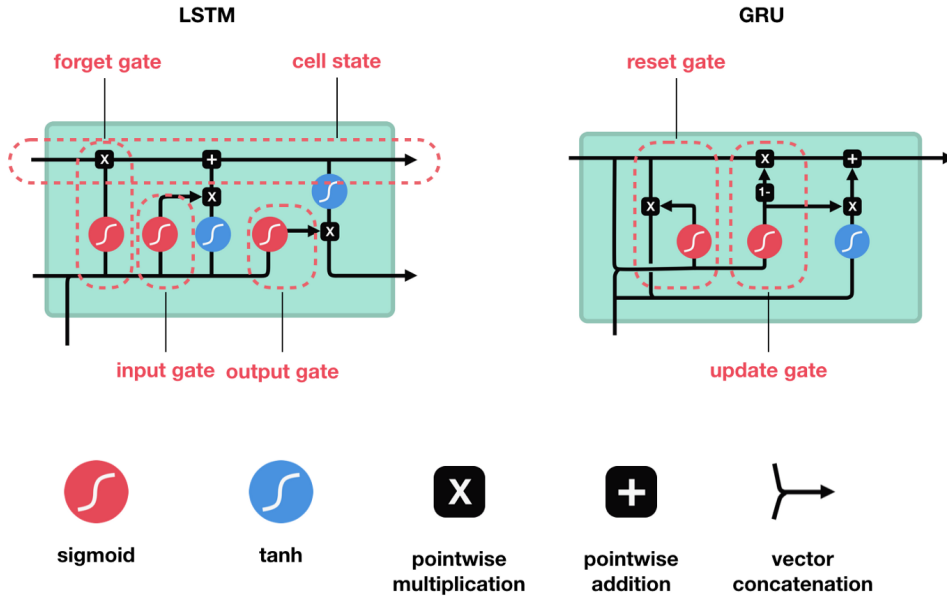


Figure 1: LSTM versus GRU

Input Gate The *input gate* updates the cell state with new information. This is achieved by passing added hidden and current state through a sigmoid and tanh functions respectively then multiplying the outputs. The sigmoid function effectively determines how important each value is (0 non-important; 1 important). The tanh function serves to regulate the network by mapping the values between -1 and 1. Because it is a point-wise multiplication, the sigmoid directly determines which values to keep from the tanh output.

Cell State Given outputs from the *forget and input gates* we can now update the internal *cell state* by first multiplying by the forget vector (which could possibly cause dropping of values in the cell state if forget vector has values close to zero). Then the output from the cell state and forget vector is then point-wise added to the outcome of the input gate which updates the cell state with values that the network deems relevant.

Output Gate Finally, for the *output gate* which determines what the next hidden state will be, we must first take the previous hidden state and input and pass it through a sigmoid function (for importance) then multiply it by the newest cell state passed through a tanh function (regularized), and this determines what information the hidden state should carry. **The final output (to the world outside the LSTM cell) is the hidden state.**

In summary, the forget gate decides what's relevant from prior steps; the input gate decides what new information is relevant to add to the cell state and the output gate determines what the next hidden state should become.

Pytorch [documentation on LSTM](#)

1.1 Your Problem: predicting a sequence

In this problem, you will predict the next values in a sequence function given history of values by training an LSTM.

Unlike previous scripts, your code will be to fill in components of a class that will be used by support code to track and predict a series of sine curves.

You will generate two LSTM cells whose hidden state dimension is 51 h_{51}, c_{51} , and your inputs will flow as follows:

$$x \rightarrow \text{LSTM}_1(x, (h_{1,t}, c_{1,t})) \rightarrow (h_{1,t}, c_{1,t}) \rightarrow \text{LSTM}_2(h_{1,t}, (h_{2,t}, c_{2,t})) \rightarrow (h_{2,t}, c_{2,t}) \rightarrow y = Ah_{2,t} + b \rightarrow y$$

```
def LSTMCELL(prev_ct, prev_ht, input):
    combine = prev_ht + input
    ft = forget_layer(combine)
    candidate = candidate_layer(combine)
    it = input_layer(combine)
    Ct = prev_ct * ft + candidate * it
    ot = output_layer(combine)
    ht = ot * tanh(Ct)
    return ht, Ct

ct = [0, 0, 0]
ht = [0, 0, 0]
for input in inputs:
    ct, ht = LSTMCELL(ct, ht, input)
```

Figure 2: LSTM Pseudo code

Your function for forward pass will take the training data, learning rate, training steps, and number of "future" steps to extrapolate into the future.

2 Problem: Variational AutoEncoder Image Generation [9pts]

AutoEncoder Autoencoders (AE) map a high-dimensional feature space to a compressed feature space (code h) then decompress to the output space of same dimension of the input space. The goal of compressing the feature space into the *code* is to make the representation of the key, essential information more compact. The component of the network that maps the high-dimensional input to the compressed *code* is termed the Encoder, and the component of the network that decompresses the code to a representative high-dimensional feature space is termed the Decoder.

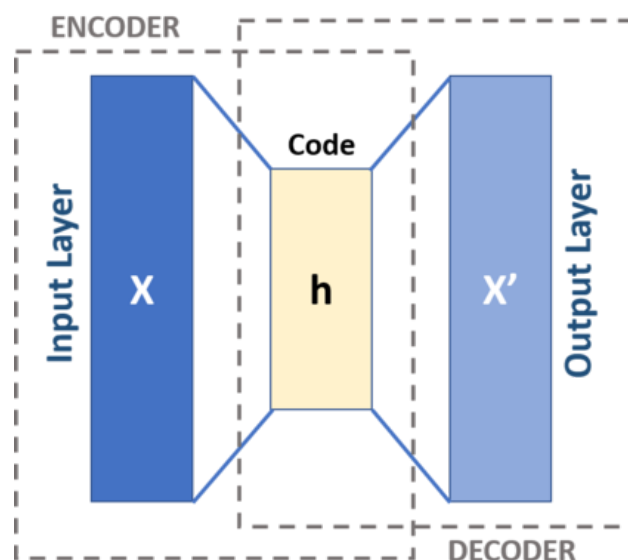


Figure 3: AutoEncoder basics ([Image source])

Now it is understood that due to compression, some information will inevitably be lost, but the essential information required to construct the *essence* of the input is retained in the *code* (h).

A practical example of this is the *de-noising* of images shown in Figure 5. which often requires the

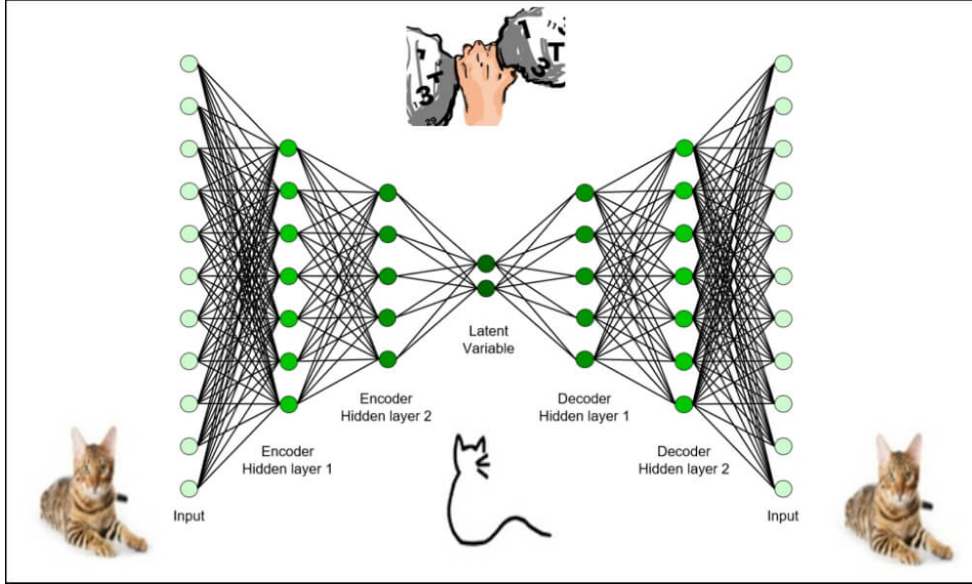


Figure 4: AutoEncoder bottleneck ([Image source])

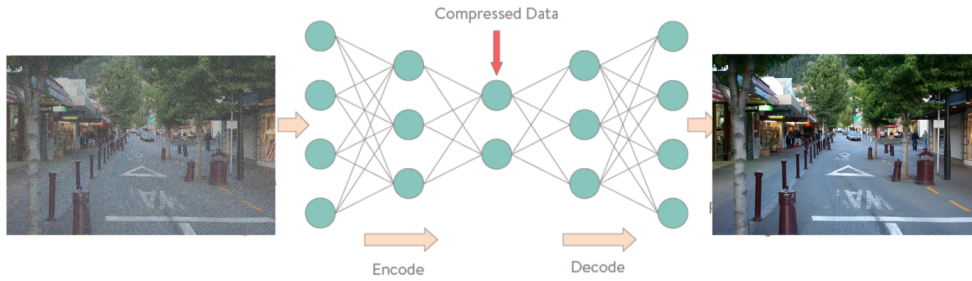


Figure 5: AutoEncoder de-noising image ([Image source])

use of Convolutional Neural Networks (CNNs) (example).

But to perform supervised training of the Autoencoder, we must first define the loss function, which will be often be the Mean Squared Error (MSE) loss between the input x and the predicted output $\hat{x}$:

$$\mathcal{L}_{MSE} = ||x - \hat{x}||^2$$

of course this loss will never be zero, but by minimizing this loss, the prediction $\hat{x}$ will retain the salient features of the input x (Figure 8).

Variational AutoEncoder Variational Autoencoder (VAE) is an extension of the autoencoder that specifically accounts for the fact that *labeled* data might not have an injective (one-to-one) mapping, but may instead may be better represented by a *distribution* that is learned in the training data. That is to say, for a given input x it is not always best to predict an output $\hat{x}$ that is simply close to the given input x , but it is also important that for the given input, the output also reflect the distribution of similar points near x that was observed in the training data.

The key concept of the VAE comes from the use of Gaussian Mixture Models (GMMs) to approximate some (multivariate) distribution $P(x)$. To achieve this, a set of Gaussians is required, and the goal is to at any point, select the appropriate Gaussian that best approximates the output of $P(x)$ at that point.

So in the autoencoder we had a *code* h , but now, we replace this *code* in the VAE with a continuous variable z which follows a standard normal distribution $N(0, 1)$ (0 mean, standard deviation of 1). So for each sample z there is an associated mean and standard deviation variables μ and σ

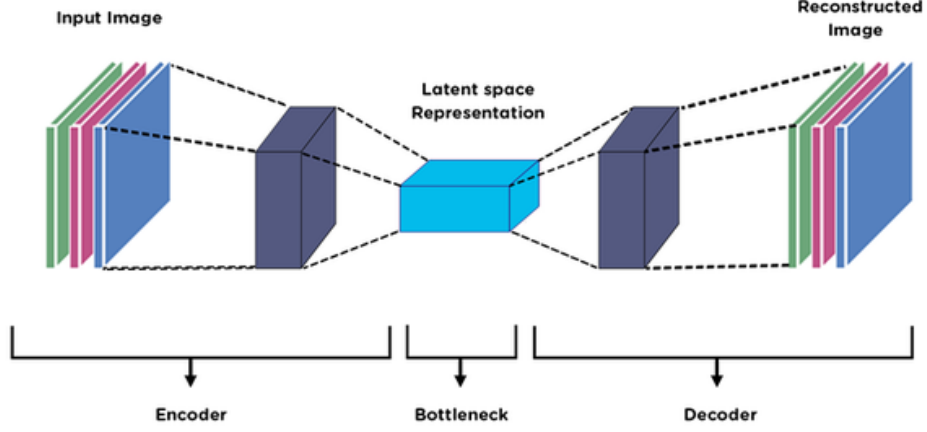


Figure 6: AutoEncoder image reconstruction ([Image source])

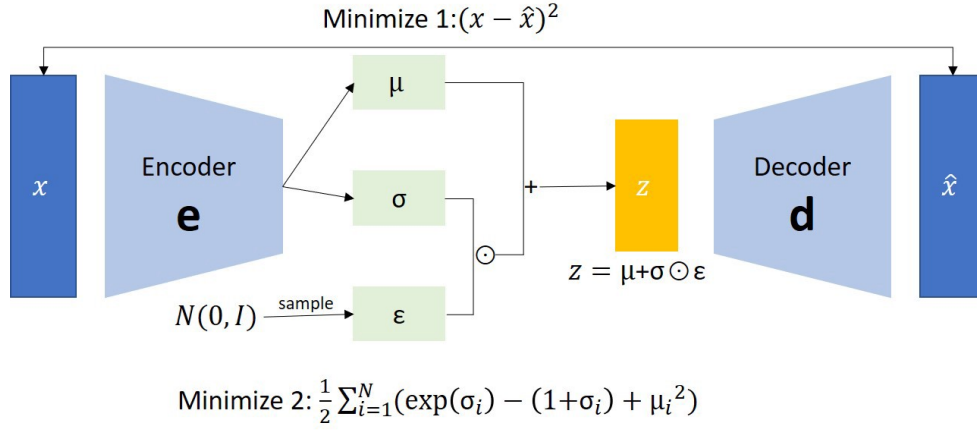


Figure 7: Variational AutoEncoder architecture ([Image source])

of the Gaussian distribution corresponding to z . Just as in the GMM, if we integrate over all of the Gaussian distributions, the integrated domain becomes the original distribution $P(x)$:

$$P(x) = \int_z P(z)P(x|z)$$

Where we have that $z \sim N(0, 1)$ (z is sample from normal distribution with 0 mean and variance of 1), $x|z \sim N(\mu(z), \sigma(z))$ (estimated variable x is conditioned on the latent variable z is sampled from a Gaussian distribution whose mean and variance are related to the continuous latent variable z). At this point, $P(z)$ is known to be $(N(0, 1))$, and the conditional probability is what must be solved for as we need to find what μ, σ should be to help us approximate $P(x)$.

Our goal is to produce results that are very similar to the input x , which means our objective is to maximize our prediction $P(x)$ (likelihood that we predict a value very similar to the input). If this is achieved on training points, then it is likely to produce similar samples when deployed.

So to maximize the $P(x)$, we can state this objective as L :

$$\max L = \sum_x \log(P(x))$$

Here we leverage the log of the likelihood ($P(x)$) to give us a nice monotonically increasing function

related to probability ($P(x) \in [0, 1]$, therefore $\log(P(x)) \in [-\infty, 0]$ where when maximized $\log(P(x)) = \log(1) = 0$).

Given the objective to maximize $\log(P(x))$, we can first expand this term as follows:

We introduce the distribution $q(z|x)$ which once summed over z produces $\log P(x)$:

$$\log P(x) = \int_z q(z|x) \log(P(x)) dz$$

We then use [Bayes Theorem](https://en.wikipedia.org/wiki/Bayes

$$= \int_z q(z|x) \log \left(\frac{P(z, x)}{P(z|x)} \right) dz$$

We then multiply the $\log(\cdot)$ argument by 1, introducing to the numerator and denominator the distribution $q(z|x)$:

$$= \int_z q(z|x) \log \left(\frac{P(z, x)q(z|x)}{q(z|x)P(z|x)} \right) dz$$

By [logarithmic identities](#) we can turn this into two integrals separating terms as follows:

$$= \int_z q(z|x) \log \left(\frac{P(z, x)}{q(z|x)} \right) dz + \int_z q(z|x) \log \left(\frac{q(z|x)}{P(z|x)} \right) dz$$

The second term is the [Kullback–Leibler \(KL\) divergence](#) which has a value greater than or equal to zero:

$$= \int_z q(z|x) \log \left(\frac{P(z, x)}{q(z|x)} \right) dz + D_{KL}(q(z|x)||P(z|x))$$

The KL divergence (also called relative entropy) is a statistical distance of how different two probability distributions are from each other, but the ordering matters for this metric. For $D_{KL}(Q||P)$ (the divergence of Q from P) is the expected excess *surprise* from using P as a model when the actual distribution is Q.

**Because the KL term is equal to or greater than 0, we've found a lower bound of the $\log P(x)$ ** (again also leveraging Bayes theorem):

$$\log P(x) \geq \int_z q(z|x) \log \left(\frac{P(x|z)P(z)}{q(z|x)} \right) dz$$

We call this the Evidence Based Lower Bound (ELBO):

$$\log P(x) \geq ELBO = \int_z q(z|x) \log \left(\frac{P(x|z)P(z)}{q(z|x)} \right) dz = E_{q(z|x)} \left[\log \frac{p(x, z)}{q(z|x)} \right]$$

The last term on the right above leverages [Expected value](#) notation $E[\cdot]$.

So we can rewrite the original form as follows:

$$\log P(x) = ELBO + D_{KL}(q(z|x)||P(z|x))$$

By adjusting $q(z|x)$ to make the ELBO higher, the KL divergence must get smaller and smaller. We continue to update $q(z|x)$ until it is the same as $P(z|x)$ making the KL divergence converge to 0, and the ELBO and $\log P(x)$ are fully consistent. Maximizing $\log P(x)$ is equivalent to maximizing the ELBO.

Adjusting $P(x|z)$ is achieved by adjusting the Decoder, and adjusting $q(z|x)$ is adjusting the encoder. So every time the decoder is updated, the encoder must be adjusted to be consistent with it so that the decoder will be continue to improve after each next training epoch.

We can further simplify our ELBO expression with the following derivation:

$$ELBO = \int_z q(z|x) \log \left(\frac{P(x|z)P(z)}{q(z|x)} \right) dz$$

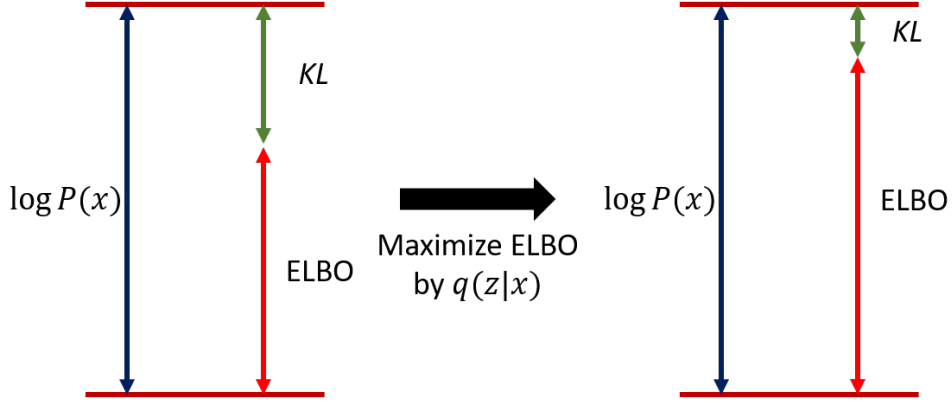


Figure 8: Variational AutoEncoder ELBO (evidence-based lower bound) ([Image source])

by logarithmic identities ($\log(ab) = \log(a) + \log(b)$ where here $a = \frac{P(z)}{q(z|x)}$, $b = P(x|z)$):

$$ELBO = \int_z q(z|x) \log \left(\frac{P(z)}{q(z|x)} \right) dz + \int_z q(z|x) \log (P(x|z)) dz$$

This can be rewritten using [KL divergence identity](#)

$$ELBO = -D_{KL}(q(z|x)||P(z)) + \int_z q(z|x) \log (P(x|z)) dz$$

Therefore, maximizing the ELBO is equivalent to minimize $D_{KL}(q(z|x)||P(z))$ (as it is a negative value) and maximizing the integral equation in the second term of the above:

$$\max_z \int_z q(z|x) \log (P(x|z)) dz = \max E_{q(z|x)}[\log P(x|z)]$$

The above expectation $\max E_{q(z|x)}[\log P(x|z)]$ means maximize the likelihood $P(x|z)$ (Decoder's output) given that $q(z|x)$ (Encoder's output) is as high as possible. This is similar to AutoEncoder's loss function (reconstruction error):

$$L = (x - \hat{x})^2$$

Finally, we can simplify $-D_{KL}(q(z|x)||P(z))$ by first decomposing its components using the logarithmic properties $\log(\frac{a}{b}) = \log(a) - \log(b)$ where $a = P(z)$ and $b = q(z|x)$, so for $P(z)$:

$$\int_z q(z|x) \log P(z) dz = \int_z N(z; \mu, \sigma^2) \log N(z; 0, I) dz$$

which for the special case of our distributions being Gaussian, is equivalent to

$$= -\frac{J}{2} \log(2\pi) - \frac{1}{2} \sum_{i=1}^J (\mu_i^2 + \sigma_i^2)$$

where J is the dimension of z , μ, σ denote the variational mean and standard deviation vectors, and μ_i, σ_i simply denote the i -th element of these vectors.

Likewise, for $q(z|x)$:

$$\int_z q(z|x) \log q(z|x) dz = \int_z N(z; \mu, \sigma^2) \log N(z; \mu, \sigma^2) dz$$

which for the Gaussian case is equivalent to the closed form:

$$= -\frac{J}{2} \log(2\pi) - \frac{1}{2} \sum_{i=1}^J (1 + \log \sigma_i^2)$$



Figure 9: MNIST dataset examples ([Image source](#))

Therefore we can write $-KL(q(z|x)||P(z))$ as:

$$-D_{KL}(q(z|x)||P(z)) = \int_z q(z|x) (\log P(z) - \log q(z|x)) dz$$

which in closed form is:

$$-D_{KL}(q(z|x)||P(z)) = -\frac{1}{2} \sum_{i=1}^J (1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2)$$

where again, μ_i, σ_i simply denote the i-th element of these vectors μ, σ .

If we use a Bernoulli or Gaussian Multi-Layer Perception (MLP) to capture the distributions $q(z|x) = q_\phi(z|x)$, $P(z) = p_\theta(z)$, $P(x|z) = p_\theta(x|z)$, then for a marginal likelihood composed of a sum over the marginal likelihoods of individual data points $\log p_\theta(x^{(1)}, \dots, x^{(N)}) = \sum_{i=1}^N \log p_\theta(x^{(i)})$ then this marginal likelihood we've seen for the i-th data point is:

$$\log p_\theta(x^{(i)}) = \underbrace{D_{KL}(q_\phi(z|x^{(i)})||p_\theta(z|x^{(i)}))}_{\text{regularization}} + \underbrace{\mathcal{L}(\theta, \phi; x^{(i)})}_{\text{loss}}$$

Then the loss term is:

$$\mathcal{L}(\theta, \phi; x^{(i)}) = \underbrace{-D_{KL}(q_\phi(z|x^{(i)})||p_\theta(z))}_{\text{regularization}} + \underbrace{\mathbb{E}_{q_\phi(z|x^{(i)})} [\log p_\theta(x^{(i)}|z)]}_{\text{reconstruction}}$$

Which simplified becomes:

$$\mathcal{L}(\theta, \phi; x^{(i)}) \simeq \frac{1}{2} \sum_{j=1}^J \left(1 + \log((\sigma_j^{(i)})^2) - (\mu_j^{(i)})^2 - (\sigma_j^{(i)})^2 \right) + \frac{1}{L} \sum_{l=1}^L \log p_\theta(x^{(i)}|z^{(i,l)})$$

where from the *reparameterization trick* we have $z^{(i,l)} = \mu^{(i)} + \underbrace{\sigma^{(i)} \odot \epsilon^{(l)}}_{\text{element-wise product}}$ and $\epsilon^{(l)} \sim \mathcal{N}(0, I)$

Good references

- [Auto-Encoding Variational Bayes](#)
- [Variational Autoencoder \(VAE\) - Roger](#)
- [Tutorial on Variational Autoencoders](#)

2.1 Your Problem: VAE for MNIST Dataset

In this problem you will leverage a VAE to perform image reconstruction for a hand-written number given an input using the MNIST dataset (Figure 9). Your output will be in distribution of the original input (an auto-generated 'hand-written' number of the same type)

3 Problem: Self-Attention for Sequence Prediction [8pts]

From Sequential to Parallel In Problem 1, we used an LSTM to process sequences *step by step*, maintaining a hidden state that serves as memory. While effective, this sequential processing has a fundamental limitation: information from early timesteps must pass through many intermediate steps to influence later predictions, which can lead to information loss (the vanishing gradient problem).

Attention offers an alternative paradigm: instead of processing sequentially, we *look at all timesteps at once* and learn which ones are most important for making each prediction. This is the core idea behind the “[Attention Is All You Need](#)” paper (Vaswani et al., 2017), which introduced the Transformer architecture that powers modern language models (GPT, BERT, etc.). Since then, transformers are a staple in the fields of robotics, NLP, computer vision, sound processing etc. It doesn’t lose ‘memory’ like LSTMs, but scales so elegantly, and has formed the foundation of ‘scaling laws’.

Scaled Dot-Product Attention The attention mechanism uses three components: **Queries** (Q), **Keys** (K), and **Values** (V). An intuitive analogy is a library search:

- **Query:** what you are looking for (your search terms)
- **Key:** the label/index on each book (what each item offers)
- **Value:** the actual content of each book (the information you retrieve)

The attention mechanism computes a weighted sum of Values, where the weight for each Value is determined by how well the corresponding Key matches the Query:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where d_k is the dimension of the key vectors. The scaling factor $\frac{1}{\sqrt{d_k}}$ is critical: without it, the dot products QK^T grow large in magnitude as d_k increases, pushing the softmax function into regions where it has extremely small gradients, effectively preventing learning.

Multi-Head Attention Rather than performing a single attention function, *multi-head attention* runs the attention mechanism multiple times in parallel with different learned linear projections of Q , K , and V . Each “head” can learn to attend to different types of relationships in the data (e.g., one head might focus on nearby timesteps while another captures long-range periodicity). The outputs of all heads are concatenated and linearly projected to produce the final result.

Good references:

- [Attention Is All You Need](#) (Vaswani et al., 2017)
- [The Illustrated Transformer](#) (Jay Alammar)
- [PyTorch MultiheadAttention documentation](#)

3.1 Your Problem: Attention-Based Sequence Prediction

In this problem, you will build a self-attention model to predict the **same sine wave sequences from Problem 1**. This allows a direct comparison between the LSTM approach (sequential memory) and the attention approach (parallel, weighted lookups, arguably simpler!).

Your model architecture is:

$$x \rightarrow \text{Linear}(1, d_{\text{model}}) \rightarrow \text{Self-Attention}(Q, K, V) \rightarrow \text{Linear}(d_{\text{model}}, 1) \rightarrow y$$

You will implement:

1. The Q , K , V projection layers

2. The `scaled_dot_product_attention` function (compute scores, apply softmax, weighted sum)
3. The forward pass wiring these components together

After training, you will visualize the **attention weight heatmap** to see which input timesteps the model “attends to” when making each prediction.

3.2 Bonus Question: Attention Beyond Text Sequences

Name one concrete example of how attention mechanisms or Transformers have been applied to a data modality *other than* text sequences (relevant to robotics, mechanical engineering, or computer science). Briefly describe what the input tokens represent and why attention is beneficial for that domain.

Hint: Think about contact force profiles, 3D point clouds, multi-agent trajectories, protein structures, audio spectrograms, etc.

4 Problem: DINOv2 Vision Feature Exploration [8pts]

Vision Transformers Vision Transformers apply the Transformer architecture (from Problem 3) to images. The key idea is to split an image into fixed-size **patches** — analogous to words in a sentence — and process them with self-attention. Each patch is linearly embedded into a vector, and these patch embeddings are processed by a standard Transformer encoder with self-attention. This allows the model to learn relationships between different spatial regions of the image.

For example, with a patch size of 14 pixels and an input image of 518×518 pixels, we obtain a $37 \times 37 = 1369$ grid of patch tokens, plus one special [CLS] token that summarizes the entire image.

DINOv2 [DINOv2](#) (Oquab et al., 2023) is a self-supervised pretrained Vision Transformer from Meta AI Research. It is trained *without any labels* using a self-distillation objective: a student network learns to match the output of a teacher network (an exponential moving average of the student) on different augmented views of the same image. The result is a model that produces rich, semantically meaningful patch-level features — patches depicting similar semantic content (e.g., two parts of a dog’s fur, or two regions of sky) will have similar feature vectors, even without any supervised training.

Patch Feature Similarity Each patch token from DINOv2 is a feature vector (e.g., 384 dimensions for ViT-S/14). We can measure how similar two patch features are using **cosine similarity**:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|}$$

which ranges from -1 (opposite directions) to 1 (identical directions). Patches with high cosine similarity represent semantically similar regions of the image.

Good references:

- [DINOv2: Learning Robust Visual Features without Supervision](#)
- [An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale \(ViT paper\)](#)
- [PyTorch Hub: DINOv2](#)

4.1 Your Problem: Interactive Feature Visualization

In this problem, you will:

1. Load a pretrained DINOv2 model (`dinov2_vits14`) using `torch.hub`
2. Extract patch token features from a sample image
3. Compute cosine similarity between a reference patch and all other patches

4. Build an interactive matplotlib visualization where mousing over the image highlights semantically similar regions via a cosine similarity heatmap
5. Generate static similarity visualizations for several reference points (for PDF submission)

You will implement the feature extraction pipeline, cosine similarity computation, and pixel-to-patch coordinate conversion.